Security Assessment
Final Report

apxUSD

February 2026

Prepared for Apyx Labs

Table of contents

Project Summary 3
Project Scope 3
Project Overview 3
Protocol Overview 4
Findings Summary 5
Severity Matrix 5
Detailed Findings 6
High Severity Issues 8
H-01 Cooldown griefing allows attacker to lock victim's funds in UnlockToken 8
Medium Severity Issues 11
M-01 Backing model is entirely trust-based with no on-chain verification 11
M-02 Deny list is not enforced on ApyUSD transfers, allowing denied addresses to circumvent restrictions 13
M-03 Partial claims corrupt the vesting schedule, allowing early token extraction 16
M-04 Yield distribution can game faithful depositors earned dividends 18
Low Severity Issues 21
L-01 setVestingPeriod enables instant share price manipulation 21
L-02 Extending rate limit period may cause temporary under-counting of minted amounts 22
L-03 Default unlocking delay in deployment config is set to test value 23
L-04 MinterV0 signatures are not EIP-712-compliant 25
L-05 UnlockToken and CommitToken do not implement ERC-165 as required by ERC-7540 26
Informational Issues 27
I-01. nonce truncation causes collisions after 256 operations 27
I-02. MinterV0 pause/unpause functions not assigned to any role 27
I-03. Deny list timing creates sanction compliance weaknesses 28
I-04. Natspec-related issues 28
Disclaimer 30
About Certora 30

Project Summary

Project Scope

Project Name	Repository (link)	Initial Commit Hash	Platform
apxUSD	apyx-labs/evm-contracts	7a61f09	EVM

Project Overview

This document describes the verification of **apxUSD** using manual code review. The work was undertaken from **January 19th, 2026** to **January 28th, 2026**.

The team performed a manual audit of the following Solidity contracts:

- `src/ApxUSD.sol`
- `src/ApyUSD.sol`
- `src/AddressList.sol`
- `src/CommitToken.sol`
- `src/LinearVestV0.sol`
- `src/MinterV0.sol`
- `src/Roles.sol`
- `src/UnlockToken.sol`
- `src/YieldDistributor.sol`
- `src/interfaces/IAddressList.sol`
- `src/interfaces/IApyUSD.sol`
- `src/interfaces/ICommitToken.sol`
- `src/interfaces/IMinterV0.sol`
- `src/interfaces/IUnlockToken.sol`
- `src/interfaces/IVesting.sol`
- `src/interfaces/IYieldDistributor.sol`
- `src/errors/AddressNotSet.sol`

- `src/errors/Denied.sol`
- `src/errors/InsufficientBalance.sol`
- `src/errors/InvalidAddress.sol`
- `src/errors/InvalidAmount.sol`
- `src/errors/InvalidCaller.sol`
- `src/errors/NotSupported.sol`
- `src/errors/SupplyCapped.sol`

During the manual audit, the Certora team discovered bugs in the Solidity contracts code, as listed on the following page.

Protocol Overview

apxUSD is a stablecoin protocol backed by off-chain preferred shares that generate dividend yields. The protocol issues apxUSD (a 1:1 backed stablecoin) and apyUSD (a yield-bearing ERC-4626 vault wrapper). Minting is performed through EIP-712 signed orders with built-in delays, enabling off-chain compliance checks before execution. Yield from preferred share dividends is distributed through a vesting mechanism that gradually releases value to apyUSD depositors over configurable periods.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	-	-
High	1	1	1
Medium	4	4	2
Low	5	5	5
Informational	4	4	3
Total	14	14	11

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
H-01	Cooldown griefing allows attacker to lock victim's funds in UnlockToken	High	Fixed
M-01	Backing model is entirely trust-based with no on-chain verification	Medium	Acknowledged
M-02	Deny list is not enforced on ApyUSD and ApxUSD transfers, allowing denied addresses to circumvent restrictions	Medium	Fixed
M-03	Partial claims corrupt the vesting schedule, allowing early token extraction	Medium	Fixed
M-04	Yield distribution can game faithful depositors earned dividends	Medium	Acknowledged
L-01	setVestingPeriod enables instant share price manipulation	Low	Fixed
L-02	Extending rate limit period may cause temporary under-counting of minted amounts	Low	Fixed
L-03	Default unlocking delay in deployment config is set to test value	Low	Fixed
L-04	MinterV0 signatures are not EIP-712-compliant	Low	Fixed

L-05	UnlockToken and CommitToken do not implement ERC-165 as required by ERC-7540	Low	Fixed
I-01	nonce truncation causes collisions after 256 operations	Informational	Fixed
I-02	MinterV0 pause/unpause functions not assigned to any role	Informational	Fixed
I-03	Deny list timing creates sanction compliance weaknesses	Informational	Acknowledged
I-04	Natspec-related issues	Informational	Fixed

High Severity Issues

H-01 Cooldown griefing allows attacker to lock victim's funds in UnlockToken		
Severity: **High**	Impact: **High**	Likelihood: **High**
Files: ApyUSD.sol#L270-L294	Status: Fixed	

Description: An attacker can permanently prevent a victim from claiming their `UnlockToken` redemption by repeatedly resetting their cooldown timer. The attack costs approximately 1 wei per grief cycle (+ gas cost).

When a user withdraws from `ApyUSD`, the contract deposits assets into `UnlockToken` and creates a redeem request on behalf of the `receiver` parameter. The receiver can be any non-denied address - it does not need to match the owner whose shares are being burned:

```
None
// ApyUSD.sol - _withdraw()
IERC4626(address($.unlockToken)).deposit(assets, receiver);
$.unlockToken.requestRedeem(assets, receiver, receiver);
```

In `CommitToken`, when a redeem request is incremented (i.e., assets are added to an existing request), the `requestedAt` timestamp is unconditionally reset:

```
None
// CommitToken.sol - _requestRedeem()
request.shares += shares;
request.assets += assets;
request.requestedAt = uint48(block.timestamp);  // Always resets cooldown
```

This allows an attacker to call `apyUSD.redeem(1 wei, victim, attacker)`, burning 1 wei of their own shares to add 1 wei to the victim's pending request and reset the victim's cooldown to the full unlocking delay (14 days by default).

The cost/impact asymmetry is extreme. Each reset costs 1 wei (~$0) plus gas (~$0.05). With 26 resets per year (every 14 days), an attacker can permanently lock a victim's funds for ~$1.30/year regardless of the amount locked. A victim with a large pending redemption (e.g., $100,000) can be permanently denied access to their funds. The attacker simply monitors the victim's cooldown and resets it before it expires. The victim cannot partially claim their original amount because `CommitToken` requires exact share matching on redemption.

Exploitation is straightforward and cheap. Any user holding even a tiny amount of apyUSD can target any other user. The attack requires no special permissions, no flash loans and no complex timing. A determined attacker could automate this indefinitely.

Users can also inadvertently grief themselves. If a user is unaware that withdrawing to an address with a pending request resets that cooldown, they may accidentally extend their own unlock period by making a second withdrawal before claiming the first.

Proof of Concept:

```
None
contract CooldownGriefingTest is ApyUSDTest {

    function test_H01_CooldownGriefingX() public {
        uint256 aliceDeposit = 100_000e18;
        depositApxUSD(alice, aliceDeposit);

        // 1. Alice withdraws, starting 14-day cooldown
        vm.prank(alice);
        apyUSD.redeem(aliceDeposit, alice, alice);

        // 2. Time passes - 1 hour remaining
        vm.warp(block.timestamp + 14 days - 1 hours);
        assertEq(unlockToken.cooldownRemaining(0, alice), 1 hours);
```

```
        // 3. Bob griefs with 1 wei
        depositApxUSD(bob, 1000e18);
        vm.prank(bob);
        apyUSD.redeem(1 wei, alice, bob);  // receiver=alice, owner=bob

        // 4. Alice's cooldown reset to full 14 days
        assertEq(unlockToken.cooldownRemaining(0, alice), 14 days);
    }
}
```

Recommendations:

1. **Do not bundle redemptions (recommended)** - Create separate requests per withdrawal instead of aggregating into a single request. This eliminates the interaction entirely and gives users clear, independent cooldowns for each withdrawal.
2. **Revert if receiver has a pending request** - In `_requestRedeem()`, revert if `request.requestedAt != 0`. This forces users to claim before making another request and prevents third parties from interfering with existing requests.
3. **Require `receiver == owner` in ApyUSD** - This prevents third parties from creating requests on behalf of others. But it still allows users to accidentally grief themselves.

Customer's response: Fixed in PR#53. `ApyUSD._withdraw()` now enforces `receiver == owner`, preventing third parties from depositing into another user's UnlockToken request and resetting their cooldown timer. Comprehensive tests were added to verify the griefing vector is blocked while normal withdrawals remain unaffected.

Fix review: Fix confirmed

Medium Severity Issues

M-01 Backing model is entirely trust-based with no on-chain verification		
Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: N/A	Status: Acknowledged	

Description: ApxUSD is a yield-bearing token backed by off-chain preferred stocks. The protocol describes an elaborate hedging strategy including Protocol Owned Liquidity (10% of TVL), cash buffers (5-25%), derivative hedges and diversification. However, the backing model is entirely trust-based as none of these mechanisms have any on-chain representation or verification.

The smart contracts contain no mechanism to verify, enforce or even observe the claimed backing. Users must trust the Foundation's claims without any ability to verify them on-chain. This is below the standard set by comparable protocols:

- Ethena: On-chain reserve fund address
- Ondo USDY: On-chain NAV oracle updated daily by trusted attester

If the protocol becomes undercollateralized, users have no on-chain warning or detection mechanism.

Recommendations: The following mechanisms can be implemented on-chain to provide verification of the claimed backing model while respecting the operational flexibility required for RWA protocols:

1. **Implement a NAV attestation oracle** - Deploy an oracle contract where a trusted attester (custodian, auditor or multi-sig) publishes signed NAV data. This follows the proven pattern used by Ondo USDY and provides daily verifiable NAV updates signed by a trusted party.

2. **Create a reserve fund address** - Designate an on-chain address holding the Protocol Owned Liquidity (POL) reserves. This is trivial to implement and provides transparency for the claimed 10% liquid buffer. Document clearly that this represents only the liquid on-chain buffer, not the full backing (which is held in off-chain preferred securities).
3. **Add `burn()` function to ApxUSD** - The protocol plans to remove tokens from circulation by sending them to dead addresses after off-chain redemptions. For `totalSupply()` to accurately reflect circulating supply, add a restricted burn function. This ensures that when the Foundation redeems apxUSD through off-chain settlement (user returns apxUSD, receives USD via wire), the tokens are properly burned. This is also essential for meaningful collateralization ratio calculations.

Customer's response: Acknowledged. We agree that verifiability of backing is important for user confidence and protocol transparency. Regarding the specific recommendations: A burn function has been implemented on `ApxUSD` via the OpenZeppelin `ERC20BurnableUpgradeable`. When the Foundation settles off-chain redemptions, the corresponding apxUSD tokens can now be properly burned on-chain.

While we are actively exploring on-chain mechanisms such as tokenized preferred share positions, collateral (NAV) oracles, and dividend oracles, we believe the most immediate and impactful step toward verifiable backing is establishing a rigorous third-party audit program for the off-chain assets collateralizing apxUSD. Independent audits of custodial holdings provide users with verifiable assurance that the claimed backing exists and is properly maintained — without requiring the full technical complexity of on-chain oracle infrastructure at launch.

This approach is well-established among RWA protocols and traditional financial products alike, where periodic third-party attestations from reputable audit firms serve as the primary trust anchor. We intend to publish these audit reports publicly and on a regular cadence, giving users and integrators a reliable mechanism to verify collateralization independent of the Foundation's own reporting.

M-02 Deny list is not enforced on ApyUSD and ApxUSD transfers, allowing denied addresses to circumvent restrictions

Severity: **Medium**	Impact: **Medium**	Likelihood: **High**
Files: ApyUSD.sol#L131-L136	Status: Fixed	

Description: The `_update()` hook in `ApyUSD` does not check the deny list, despite the natspec stating it "Enforces pause, freeze, and deny list functionality". As a result, denied addresses can bypass restrictions by transferring shares to a clean address. Additionally, the deny list is not checked on apxUSD transfers either despite the `AddressList` NatSpec stating that it should serve as a "single source of truth for blocked/allowed addresses across all Apyx contracts".

The ApyUSD contract implements two separate restriction mechanisms: a freeze list (checked in `_update()` via `ERC20FreezeableUpgradable`) and a deny list (checked only in `_deposit()` and `_withdraw()`). While frozen addresses cannot transfer tokens, denied addresses can. This inconsistency allows denied users to move their shares freely. A denied address holding `apyUSD` shares can transfer them to another address they control. That clean address can then call `withdraw()` and receive the underlying assets. The deny list, intended as a compliance control, provides no meaningful restriction once a user already holds shares.

As for the ApxUSD contract - it does not reference and integrate the `AddressList` in any way. And thus any ApxUSD transfer is allowed even between addresses that are part of the protocol deny list.

The likelihood is high. Any denied user with existing shares can execute this bypass trivially. The impact depends on why addresses are denied - if it's for sanctions compliance or regulatory reasons, this circumvention could expose the protocol to legal risk.

Proof of Concept:

The following test is a proof-of-concept. It should be added to `ApyUSD.t.sol`:

```
None
function test_DenyListBypassViaTransfer() public {
   // 1. Alice deposits and receives shares
   uint256 depositAmount = 1000e18;
   deal(address(apxUSD), alice, depositAmount);

   vm.startPrank(alice);
   apxUSD.approve(address(apyUSD), depositAmount);
   apyUSD.deposit(depositAmount, alice);
   vm.stopPrank();

   uint256 aliceShares = apyUSD.balanceOf(alice);
   assertGt(aliceShares, 0, "Alice should have shares");

   // 2. Alice gets added to deny list
   vm.prank(admin);
   denyList.add(alice);

   // 3. Alice cannot withdraw directly (expected)
   vm.prank(alice);
   vm.expectRevert();
   apyUSD.withdraw(depositAmount, alice, alice);

   // 4. But Alice **can** transfer shares to Bob (bypasses the deny list)
   vm.prank(alice);
   apyUSD.transfer(bob, aliceShares);

   assertEq(apyUSD.balanceOf(bob), aliceShares, "Bob received shares");
   assertEq(apyUSD.balanceOf(alice), 0, "Alice has no shares");

   // 5. Bob can withdraw (he is not on deny list)
   vm.prank(bob);
   apyUSD.withdraw(depositAmount, bob, bob);
```

```
    // 6. Alice's funds successfully exited via Bob, bypassing deny list
    assertGt(unlockToken.balanceOf(bob), 0, "Bob has unlock tokens");
}
```

Recommendations: Check the deny list in `_update()` for both sender and receiver:

```
None
 function _update(address from, address to, uint256 value)
      internal
      override(ERC20Upgradeable, ERC20PausableUpgradeable,
ERC20FreezeableUpgradable)
  {
      ApyUSDStorage storage $ = _getApyUSDStorage();
      if (from != address(0)) _revertIfDenied($, from);
      if (to != address(0)) _revertIfDenied($, to);
      super._update(from, to, value);
  }
```

Customer's response: Fixed in PR#44. The freeze mechanism was replaced with a centralized `ERC20DenyListUpgradable` extension inherited by both `ApxUSD` and `ApyUSD`. Deny list checks are now enforced in `_update()` for all transfers (including `transfer()` and `transferFrom()`), preventing denied addresses from moving tokens to clean addresses.

Fix review: Fix confirmed

M-03 Partial claims corrupt the vesting schedule, allowing early token extraction

Severity: **Medium**	Impact: **Medium**	Likelihood: **High**
Files: [LinearVestV0.sol#L157-L162](#)	Status: Fixed	

Description: The contract's vesting calculation lets users withdraw tokens faster than intended after a partial claim. This happens because the function that calculates how many tokens have vested does not track past withdrawals correctly.

The vesting formula is:

```
None
(vestingAmount * timeSinceLastDeposit) / vestingPeriod
```

When a beneficiary claims some vested tokens, the contract reduces the internal `vestingAmount` variable but does not reset the vesting clock (`lastDepositTimestamp`):

```
None
function transferVestedYield() external override onlyBeneficiary {
    uint256 vested = vestedAmount();
    vestingAmount -= vested;

    _transferVestedYield(vested);
}
```

As time goes on, new claims apply the vesting formula to this reduced amount, effectively compounding the vested share. This means a beneficiary can extract more tokens than they should at each step, ending up with more tokens earlier than the vesting schedule allows.

The total amount received over the full vesting period remains correct - the beneficiary cannot extract more tokens than allocated. However, the vesting contract exists to enforce when tokens become available and this invariant is broken.

Recommendations: Track the original deposit amount and claimed amount separately:

```
None
function vestedAmount() public view returns (uint256) {
    if (totalDeposited == 0) return 0;

    uint256 elapsed = block.timestamp - lastDepositTimestamp;
    if (elapsed >= vestingPeriod) {
        return totalDeposited - totalClaimed;
    }

    uint256 totalVested = (totalDeposited * elapsed) / vestingPeriod;
    if (totalVested <= totalClaimed) return 0;
    return totalVested - totalClaimed;
}
```

This ensures the vesting calculation always references the original deposit amount and accounts for previous claims correctly.

Customer's response: Fixed in PR#43. The vesting logic now tracks a `fullyVestedAmount` state variable for amounts that have fully vested but not yet been transferred, and uses `lastTransferTimestamp` for precise vesting calculations. This ensures partial claims do not compound the vested share across subsequent withdrawals, preserving the intended vesting schedule.

Fix review: Fix confirmed

M-04 Yield distribution can game faithful depositors earned dividends

Severity: **Medium**	Impact: **Medium**	Likelihood: **High**
Files: ApyUSD.sol#L131-L136	Status: Acknowledged	

Description: The protocol's yield distribution creates an economic misalignment between how dividends are earned and distributed. Preferred share dividends are earned based on prior holdings, but distributed on-chain based on current holdings.

This mismatch allows new depositors to enter the vault after dividends have been earned and capture a proportional share of yield they did not generate. Therefore, existing depositors who are the rightful owners of the yield face continuous dilution of their earned yield by future depositors. This cycle continues as the vault becomes more and more profitable, where profit is unfairly diluted to holders who did not generate that value.

When considering RWAs vs web3 yield bearing vaults or staking rewards, there are distinct differences. Web3 involves some work done, a yield bearing event and then distribution. In Apyx there is an off chain yield distribution from preferred shares, then at a later date another on-chain distribution (linearly spread). The gap between those two events allows the same capital to take profit twice from essentially the same yield bearing event.

Example:

Assumption: For simplicity, $1 USD held off-chain in preferred shares is equivalent to 1 ApxUSD on-chain.

1. Holder A has 1,000,000 ApxUSD deposited in the ApyUSD vault for the month.
2. Holder B holds 1,000,000 in preferred shares directly off-chain .
3. Both Vault and Holder B earn 50,000 in monthly dividends paid directly off-chain.

4. Before the vault's yield is distributed on-chain, Holder B decides to enter the vault with 1,050,000 ApyUSD.
5. Vault state before on-chain yield distribution:
    - Holder A: 1,000,000 ApxUSD (48.78%)
    - Holder B: 1,050,000 ApxUSD (51.22%)
    - Total: 2,050,000 ApxUSD
6. Holder A's 50,000 yield is deposited via YieldDistributor.depositYield() and vests through LinearVestV0. Distribution is based on current share ownership.
7. After 28 days yield is distributed:
    - Holder A receives: 24,390 ApxUSD (48.78% of 50,000)
    - Holder B receives: 25,610 ApxUSD (51.22% of 50,000)
8. Final outcome:
    - Holder A: 24,390 total yield (lost 25,610 of their earned dividend)
    - Holder B: 50,000 off-chain + 25,610 on-chain = 75,610 total yield
9. Holder A was penalized 25,610 by investing directly through the vault.

To reduce the impact the team has implemented a 28-day linear vesting period, designed to smooth yield distribution and discourage short-term deposit-withdraw cycles. However, this is not a prevention mechanism, it only slows the dilution down.

Recommendations: Implement time-weighted yield distribution where yield is allocated based on holding duration during the dividend period, or establish a minimum holding period before new deposits become eligible for yield.

Customer's response: Acknowledged. We recognize that the current yield distribution model creates a theoretical opportunity to game yield. The protocol currently mitigates this through two complementary mechanisms:

1. A vesting period on deposited yield, which reduces the profitability of short-term deposit cycles by requiring depositors to remain in the vault for the full vesting duration to capture yield.
2. An unlocking cooldown on withdrawals, which means depositors who enter to capture yield must forfeit yield during their exit period, creating a natural economic counterbalance.

Together, these mechanisms significantly narrow the window for opportunistic behavior and make single-cycle gaming less profitable under normal operating conditions. We also note that the Foundation controls the timing and cadence of on-chain yield deposits, providing an additional operational lever to manage distribution timing relative to known deposit activity.

Low Severity Issues

L-01 setVestingPeriod enables instant share price manipulation		
Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: LinearVestV0.sol#L21	Status: Fixed	

Description: Admin can change vestingPeriod at any time, causing `vestedAmount()` to instantly jump up or down. Since ApyUSD's `totalAssets()` includes `vestedAmount()`, any change in vesting period instantly changes share prices without any token movement or time passing.

Rapid changes in exchange rate may cause unfair ApyUSD token mints/burns for users not using slippage based depositForMinShares or withdrawForMaxShares (same for mint and redeem).

Example:

1. 1000 tokens deposited 50 days ago
   vestingPeriod=100, vestedAmount=500
2. Changing vestingPeriod to 50 instantly
   vestedAmount=1000 (fully vested), inflating `totalAssets` and share price by 500 tokens worth.

Recommendations: We recommend only allowing changes to the vesting period when vesting periods for previous deposits have finished.

Customer's response: Fixed in c263465. The vesting logic was simplified to change how vested and unvested yield is represented, preventing setVestingPeriod() from causing instant jumps in vestedAmount(). An ApyUSDRateView was also added for external rate queries.

Fix review: Fix confirmed.

L-02 Extending rate limit period may cause temporary under-counting of minted amounts

Severity: **Low**	Impact: **Low**	Likelihood: **Medium**
Files: MinterV0.sol#L363-L377	Status: Fixed	

Description: The `mintHistory` queue is pruned during each `requestMint()` call based on the current `rateLimitPeriod`. If an admin extends the period (e.g., from 1 week to 2 weeks), historical entries already purged under the old period are lost. As a result, the rate limit calculation will under-count recent mints, temporarily allowing more minting than the new limit intends. For example:

1. Mints from day 1-7 exist in `mintHistory` with a 1-week rate limit period.
2. On day 8, a `requestMint()` prunes day 1 entries.
3. The admin then extends the period to 2 weeks.

The new window should include days 1-8 but day 1 data is gone. `rateLimitMinted()` returns less than actually minted, allowing excess minting until the window fully refreshes.

Recommendations: Consider implementing a cumulative tracking approach that does not lose historical data on period changes.

Alternatively, document that extending `rateLimitPeriod` may under-count minted amounts in some cases.

Customer's response: Fixed in 291241c. The mint history queue now uses a longer cutoff to prevent records from being prematurely cleared when the rate limit period changes, preserving historical mint data across period adjustments. Input validation was added to `setRateLimit()` and `rateLimitMinted()` was optimized to iterate newest-to-oldest with early termination.

Fix review: Fix confirmed

L-03 Default unlocking delay in deployment config is set to test value

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: [DeployBase.sol#L35-L36](#)	Status: Fixed	

Description: The default unlocking delay in the deployment base script is set to 1 day with a comment indicating it's for testing, but this value would be used in production deployments if not explicitly overridden.

```
None
/// @notice Default unlocking delay: 1 day for testing (86400 seconds)
uint48 public constant DEFAULT_UNLOCKING_DELAY = 1 days;
```

The test suite uses a 14-day delay (BaseTest.sol:83):

```
None
uint48 public constant UNLOCKING_DELAY = 14 days;
```

The documentation recommends an even longer period (overview.md:51):

> *apyUSD should likely implement a 28 day unstaking period, which will roughly bring the unstaked apyUSD to the next dividend of the underlying where the dividend is paid monthly, helping to support the price.*

The unlocking delay serves as a security mechanism to prevent short-term deposit-withdraw arbitrage and to align unstaking with dividend cycles. A 1-day delay significantly weakens this protection compared to the intended 14-28 day period.

If a deployment proceeds without overriding this constant, users could enter and exit positions within 1 day rather than the intended multi-week period, undermining the yield distribution design.

Recommendations: Update the default to match the intended production value:

```
None
/// @notice Default unlocking delay: 14 days (1209600 seconds)
uint48 public constant DEFAULT_UNLOCKING_DELAY = 14 days;
```

Customer's response: Fixed in 049ee5f. The deployment configuration for UnlockToken was updated to use the production unlocking delay value instead of the 1-day test default.

Fix review: Fix confirmed

L-04 MinterV0 signatures are not EIP-712-compliant

Severity: **Low**	Impact: **Low**	Likelihood: **High**
Files: [MinterV0.sol#L150-L152](#)	Status: Fixed	

Description: The `MinterV0` contract inherits OpenZeppelin's `EIP712`, initializes it in the constructor and documents `hashOrder()` as returning "the EIP-712 typed hash". However, `validateOrder()` never calls `_hashTypedDataV4()` to include the domain separator:

```
None
bytes32 digest = hashOrder(order); // Bare struct hash
address signer = digest.recover(signature); // Missing domain separator
```

EIP-712 requires signatures over `keccak256("\x19\x01" || domainSeparator || structHash)`. The domain separator binds signatures to a specific contract address and chain ID. Without it, the implementation is non-compliant. Standard wallets like MetaMask use `signTypedData_v4` which includes the domain separator. Signatures produced this way will fail validation against this contract.

Recommendations: Use the inherited `_hashTypedDataV4()` function:

```
None
bytes32 structHash = hashOrder(order);
bytes32 digest = _hashTypedDataV4(structHash);
address signer = digest.recover(signature);
```

Additionally, update the frontend and tests to sign proper EIP-712 typed data.

Customer's response: Fixed in PR#41. `validateOrder()` now wraps the struct hash with `_hashTypedDataV4()` to include the domain separator, and struct hashing uses `abi.encode()`

instead of `abi.encodePacked()` per the EIP-712 specification. A new `structHashOrder()` function was added and `DOMAIN_SEPARATOR()` was exposed publicly for off-chain signing compatibility with standard wallets.

Fix review: Fix confirmed

L-05 UnlockToken and CommitToken do not implement ERC-165 as required by ERC-7540

Severity: **Low**	Impact: **Low**	Likelihood: **High**
Files: CommitToken.sol#L16	Status: Fixed	

Description: `CommitToken` declares that it implements `IERC7540Redeem` and `IERC7540Operator`. The ERC-7540 specification states:

> *Smart contracts implementing this Vault standard MUST implement the ERC-165 supportsInterface function.*

Neither `CommitToken` nor `UnlockToken` implement `supportsInterface()`. This means other protocols and integrations using ERC-165 to detect ERC-7540 support will not recognize these contracts as compliant, breaking composability with ERC-7540-aware tooling.

Recommendations: Implement the ERC-165 standard:

```
None
function supportsInterface(bytes4 interfaceId) public view virtual returns (bool) {
    return interfaceId == type(IERC7540Redeem).interfaceId
    || interfaceId == type(IERC7540Operator).interfaceId
    || interfaceId == type(IERC165).interfaceId;
}
```

Customer's response: Fixed in PR#51. CommitToken now inherits OpenZeppelin's `ERC165` and implements `supportsInterface()`. UnlockToken inherits this from CommitToken. Integrators can now detect interface compliance via standard ERC-165 queries.

Fix review: Fix confirmed

Informational Issues

I-01. nonce truncation causes collisions after 256 operations

Description: The `_encodeOrderData()` function prepares the calldata that gets sent to OpenZeppelin's AccessManager for scheduling delayed mint operations. It encodes a call to `apxUSD.mint(beneficiary, amount)` and appends a single byte derived from the order's nonce (nonce % 256) to create a unique identifier. The nonce byte is intended to differentiate multiple orders with the same beneficiary and amount, but because it's truncated to 1 byte, it only provides uniqueness for 256 operations before colliding.

As mentioned, `_encodeOrderData()` function uses `bytes1(uint8(order.nonce % 256))` to append uniqueness to the encoded data. This truncates the `uint48 nonce` to just 1 byte, meaning only 256 unique values are possible per (beneficiary, amount) pair. After 256 operations with identical beneficiary and amount, the operationId will collide with an earlier operation. If that earlier operation is still pending in AccessManager (not executed, not cancelled, not expired), the new request will revert with AccessManagerAlreadyScheduled.

The impact is low here because failing mints require the same beneficiary amount and must remain 256 operations apart whilst old operations must still be pending.

Recommendations: Do not truncate the nonce, this removes the uniqueness guarantees unnecessarily.

Customer's response: Fixed in dec1c24. The minter nonce was extended from a 1-byte truncated value to a full `uint256`, and the nonce is now passed through to `ApxUSD.mint()` to generate unique operation IDs, eliminating the possibility of collisions after 256 operations.

Fix review: Fix confirmed

I-02. MinterV0 pause/unpause functions not assigned to any role

Description: The assignAdminTargetsFor(MinterV0) function only configures `setMaxMintAmount()` and `setRateLimit()` selectors for `ADMIN_ROLE` (As confirmed expected behaviour by documentation). The `pause()` and `unpause()` functions in MinterV0 are

marked `restricted` but are not assigned to any role. Unlike `ApxUSD` and `ApyUSD` which have `pause` and `unpause` properly configured, `MinterV0` pause functions will revert with `AccessManagerUnauthorizedCall` when called, leaving the minter unable to be paused in an emergency unless manually configured outside of `Roles.sol`.

Recommendations: Add the pause/unpause functions to the set of callable administrator restricted functions.

Customer's response: Fixed in PR#49. `MinterV0.pause.selector` and `MinterV0.unpause.selector` were added to the `ADMIN_ROLE` configuration in `Roles.sol`, matching the pattern already used for ApxUSD, ApyUSD, and CommitToken. Tests were added to verify admin can pause/unpause and non-admin calls revert.

Fix review: Fix confirmed

I-03. Deny list timing creates sanction compliance weaknesses

Description: Users denied at different stages of the withdrawal flow experience asymmetric economic outcomes. When a user is added to the deny list while holding apyUSD shares, they cannot withdraw but continue to accrue yield proportional to their share ownership. In contrast, if denial occurs after withdrawal (at UnlockToken stage), the user has already exited the yield-bearing vault, so valid users' yield is unaffected.

This provides yield bearing opportunities to sanctioned users and accounts that have not yet been added to the Deny List. If bugs are discovered later that allow withdrawal, this could lead to bypass of sanctions to earn profit, where users should not be allowed.

Recommendations: Consider ejecting users from yield bearing opportunities for any period where they are sanctioned.

Customer's response: Acknowledged. The ApyUSD contract is designed to be upgradeable, and future upgrades may introduce functionality to redeem or burn ApyUSD shares held by deny-listed addresses.

I-04. Natspec-related issues

Description:

1. The comment for the `APXUSD_STORAGE_LOC` says the hash has been derived from `"prefusd.storage.ApxUSD"` but the actual hash was derived from `"apyx.storage.ApxUSD"` . The comment is likely a leftover from a refactoring/renaming.
2. Line 64 in CommitToken.sol references the `STAKE_STRAT` role; this role doesn't exist.

Recommendations: Update the comments to reflect correct definitions, namespaces and state of documentation.

Customer's response: Fixed in f314b90. The `APXUSD_STORAGE_LOC` comment was corrected to reference the actual namespace (`apyx.storage.ApxUSD` instead of the stale `prefusd.storage.ApxUSD`), and the incorrect `STAKE_STRAT` role reference in CommitToken was removed.

Fix review: Fix confirmed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

The review was conducted over a limited timeframe and may not have uncovered all relevant issues or vulnerabilities.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.